

sabinaHSBM Docker Setup

This guide will help users run the *sabinaHSBM* package within a Linux-based Docker container.

What's included in the Docker Image

- **Linux operating system:** Ubuntu 24.04
- **R Environment:** Pre-configured with all necessary packages, including `reticulate` for Python integration.
- **Python & graph-tool:** Python is pre-configured with `graph-tool` for network analysis.
- **sabinaHSBM:** The Docker image includes the *sabinaHSBM* package and all required dependencies.
- **RStudio Server:** Installed and ready to use as a browser-based R interface.
- **Jupyter Notebook:** Pre-installed and ready to launch from the container for interactive coding and analysis.

Step-by-step setup

1. Install Docker Desktop

Download and install Docker Desktop from <https://www.docker.com/products/docker-desktop>. Registration and sign-in are required.

- **Windows users:** After installation, you can open it and run the following commands either from its integrated terminal (> **Terminal** button) or from the Windows Command Prompt (CMD) or PowerShell.
- **Mac or Linux users:** After installation, open the application by clicking the icon. Then, just open the terminal and run the given commands below.

2. Pull the docker image

To download the pre-configured image from Docker Hub, run:

```
docker pull herlima/sabinahsbm
```

Important for macOS with Apple Silicon CPUs

Macs with M-series CPUs (arm64 architecture) need to specify the amd64 platform to enable platform emulation because the image is built for amd64 architectures:

```
docker pull --platform linux/amd64 herlima/sabinahsbm
```

Docker Desktop enables platform emulation by default. However, in some cases, the emulation can cause resource heavy applications like RStudio Server to have performance issues. If you face performance issues, you can try to solve them by enabling Rosetta emulation:

Settings > General > Select “Apple Virtualization Framework” > Check the box for “Use Rosetta for x86/amd64 emulation on Apple Silicon”.

Also you should ensure that enough RAM is allocated by heading to:

Settings > Resources and select at least 4GB RAM, but more is better.

Additionally, we note that macOS is a Unix system and thus native installation of *graph-tool* python module is supported and installation instructions are available at <https://graph-tool.skewed.de/installation.html>. By installing *graph-tool* locally, users could then install *sabinaHSBM* without the need for a docker container.

3. Create and start the container

The following command creates and runs a new Docker container named `sabinahsbm_container` using the `sabinahsbm` image.

- The `-p` flags are optional and map ports 8787, 8880 from the container to the host machine, enabling access to RStudio Server and Jupyter Notebook, respectively, if needed.
- The `-v` flag is also optional (but recommended) and establishes a bind mount between your local directory (`/absolute/path/to/your/project`) and the container’s bind-mounted directory (`/home/my_project`), allowing seamless file synchronization. The path on your local machine must be an absolute path (e.g., `/Users/yourname/Documents/project` or `C:/Users/yourname/Documents/project`). Files created or modified in the bind-mounted directory will be directly accessible on your local machine. Alternatively, you can use a Docker volume instead of a bind mount. Unlike bind mounts, which link directly to local directory, volumes are stored within a Docker filesystem.

Note for Windows Users: The command below uses a backslash (`\`) to split the code across multiple lines for readability. This works in Mac/Linux terminals. However, if you are using Windows PowerShell, you must replace the backslash (`\`) with a backtick (```). If you are using the Windows Command Prompt (CMD), replace it with a caret (`^`). Alternatively, you can paste the entire command as a single continuous line without any slashes.

```
docker run -it --name sabinahsbm_container \
-p 8787:8787 -p 8880:8880 \
```

```
-v "/absolute/path/to/your/project:/home/my_project" \
herlima/sabinahsbm bash
```

For macOS with Apple Silicon

Use the `--platform` flag to run the container under amd64 emulation.

```
docker run --platform linux/amd64 -it \
  --name sabinahsbm_container \
  -p 8787:8787 -p 8880:8880 \
  -v "/absolute/path/to/your/project:/home/my_project" \
  herlima/sabinahsbm bash
```

After executing this command, you will enter an interactive shell, providing direct access to the container's command line.

4. Run R, RStudio Server or Jupyter Notebook

Once the docker container is running, you can interact with the ***sabinaHSBM*** package using different environments: through the R console, RStudio Server, or Jupyter Notebook.

Run R from the terminal

Start an R session by typing `R`. This opens an interactive console where you can load and use ***sabinaHSBM*** from the terminal environment.

Use RStudio Server (*optional*)

RStudio Server is already installed in the image.

A default user was created with username `test` and password `sabinahsbm`. But you can set your own. To create a user and password (This step is only needed the first time) (replace `yourname` with your desired username and password):

```
useradd -m yourname
passwd yourname # you will be prompted to set a password
```

Then launch RStudio Server:

```
/usr/lib/rstudio-server/bin/rsserver --server-daemonize=0 \
  --www-port=8787 --www-address=0.0.0.0
```

In your browser, open: `http://localhost:8787`

Log in with the user and password you just created, or use the default credentials.

To work in your mounted project directory `/home/my_project` use in the R console `setwd("/home/my_project");` or go to the Files pane (bottom-right), click "... " → "Go to folder..." and enter `/home/my_project`.

Note: RStudio Server runs in the foreground. Keep the terminal open while working.

Troubleshooting for Apple Silicon Users:

If RStudio Server times out or freezes indefinitely on the loading screen, your Mac is likely struggling with unoptimized emulation. Try to enable “Rosetta emulation” and ensure that Docker has at least “4GB of RAM” allocated in your Docker Desktop settings. Check Step 2 of this tutorial for more details.

Workaround: If RStudio still refuses to load, you can bypass it entirely by using the standard R terminal (type R in the console) or launching the Jupyter Notebook (detailed below). Both are lightweight and should run perfectly under emulation.

Use Jupyter Notebook *(optional)*

You can also use a Jupyter Notebook. Launch it with:

```
jupyter notebook --allow-root --no-browser \  
--ip 0.0.0.0 --port=8880
```

The terminal will display a clickable URL. Simply click the provided link or paste it into your browser to access the Jupyter interface, ideal for coding, data visualization, and interactive analysis.

5. Use the *sabinaHSBM* package

Here’s an example to get started:

```
setwd("/home/my_project")  
  
# Load sabinaHSBM  
library(sabinaHSBM)  
  
# Load the data  
data(dat, package = "sabinaHSBM")  
  
# Prepare the input  
myInput <- hsbm.input(  
  dat,  
  n_folds = 10  
)  
  
# Generate link predictions  
myPred <- hsbm.predict(myInput,  
  iter = 1000,  
  method = "conditional_missing")
```

```

)

# Reconstruct the network and evaluate
myReconst <- hsbm.reconstructed(myPred,
                                rm_documented = TRUE,
                                threshold = "prc_closest_topright")

summary(myReconst)

# Exit R
q()

```

6. Exit the Container

To exit the container, simply type:

```
exit
```

7. Stop and Restart the container

To stop the container:

```
docker stop sabinahsbm_container
```

To start the container again and access its shell, run:

```
docker start sabinahsbm_container
docker exec -it sabinahsbm_container bash
```